

Perfect. Below is **Module 4 - Knowledge Retrieval Engine Notes (Response 1)**. These are written like professional engineering notes, not like a tutorial.

Company Brain

Module 4 - Knowledge Retrieval Engine

Engineering Notes (Response 1)

Table of Contents

1. Module 4 Overview
 2. Why Module 4 Exists
 3. Position in Company Brain Architecture
 4. Policy Engine
 5. Query Intelligence
 6. Knowledge Execution Planner
 7. Parallel Retrieval
 - Dense Retrieval
 - BM25 Retrieval
 - Knowledge Graph Retrieval
 8. Hybrid Fusion Engine
 9. Evidence Orchestrator
 10. Cross Encoder Reranker
 11. Knowledge Synthesizer
-

1. Module 4 Overview

What is Module 4?

Module 4 is the **Enterprise Knowledge Retrieval Engine** of Company Brain.

Its responsibility is **not** to generate answers. Instead, it transforms a user's natural language question into a **trusted, structured, evidence-backed KnowledgeContext** that downstream modules (Chat, AI Agents, Dashboards, APIs, Workflows) can consume.

Unlike traditional Retrieval-Augmented Generation (RAG), Module 4 emphasizes:

- Multi-source retrieval
 - Knowledge graph reasoning
 - Evidence validation
 - Trust scoring
 - Provenance
 - Auditability
 - Multi-tenant security
-

Primary Goal

Convert:

User Question

into

KnowledgeContext

where KnowledgeContext contains:

- Verified evidence
 - Entities
 - Relationships
 - Timeline
 - Risks
 - Decisions
 - Citations
 - Confidence
 - Metadata
-

Module 4 Responsibilities

- Understand user intent.
- Plan retrieval strategy.
- Search multiple knowledge sources.
- Merge retrieval results.
- Organize evidence.
- Improve ranking accuracy.
- Build structured organizational knowledge.
- Prepare trusted context for downstream modules.

What Module 4 Does NOT Do

Module 4 does not:

- Generate natural language answers.
- Execute workflows.
- Call GPT/Claude/Gemini.
- Modify enterprise data.
- Perform autonomous actions.

Those responsibilities belong to later modules.

2. Why Module 4 Exists

Enterprise data is scattered across many systems:

- Slack
- Microsoft Teams
- Google Drive
- Jira
- Confluence
- CRM
- ERP
- Emails
- Internal databases
- PDFs
- Knowledge Graph

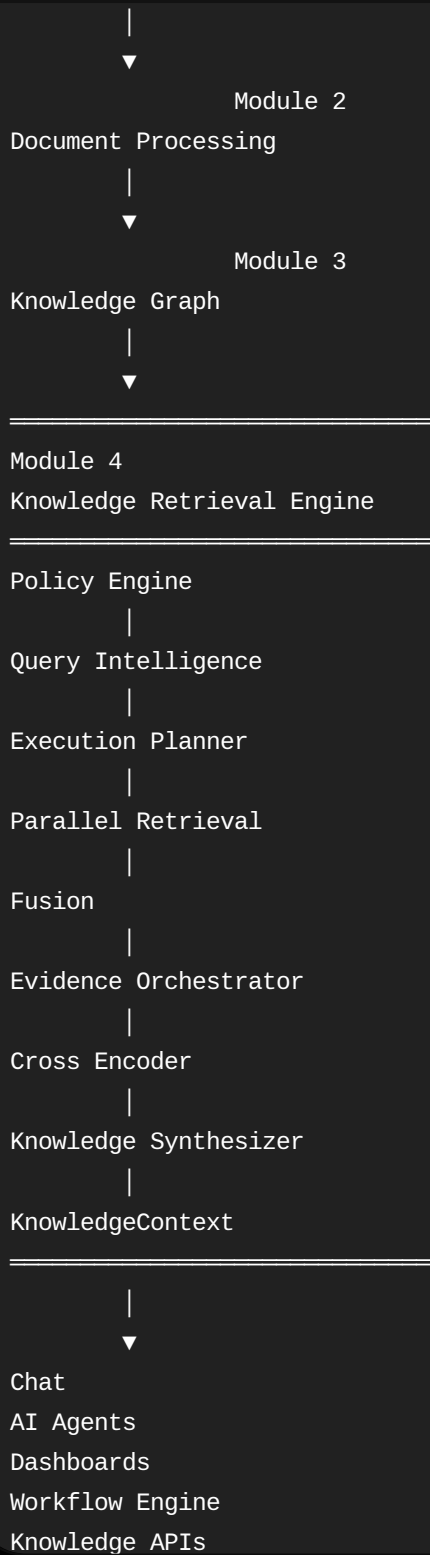
If every AI application queried these systems independently:

- Results would be inconsistent.
- Duplicate work would occur.
- Security policies would be difficult to enforce.
- Performance would suffer.

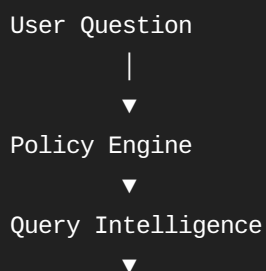
Module 4 centralizes retrieval and provides a single, trusted knowledge interface.

3. Position in Company Brain Architecture

Module 1
Enterprise Data Ingestion



High-Level Flow



Execution Planner



Dense Retrieval

BM25 Retrieval

Graph Retrieval



Fusion



Evidence Orchestrator



Cross Encoder



Knowledge Synthesizer



KnowledgeContext

4. Policy Engine

Purpose

The Policy Engine is the security gate of Module 4.

Every request must pass through it before any retrieval occurs.

Its responsibilities include:

- Authentication verification
- Tenant isolation
- Access control
- Permission validation
- Row-level security enforcement
- Query policy enforcement

Why It Exists

Suppose two companies use Company Brain:

Tenant A: Microsoft

Tenant B: Amazon

If retrieval ignores tenant boundaries, confidential information could leak across organizations.

The Policy Engine prevents this.

Responsibilities

Authentication

Validate:

- JWT
 - OAuth token
 - Session
 - API key
-

Tenant Isolation

Every request includes:

Tenant ID

Every retrieved object must belong to the same tenant.

Permission Validation

Example:

Employee

↓

Cannot access

↓

Executive salary records

Policy Engine blocks retrieval before search begins.

Row-Level Security

Every SQL query automatically filters by tenant and permissions.

Example:

SQL

SELECT *

FROM documents

WHERE tenant_id = :tenant

Query Policy

Example:

User asks:

Delete all employee records.

Module 4 is retrieval-only.

Policy Engine rejects write operations.

Output

Produces:

Authorized Request

or

Rejected Request

5. Query Intelligence

Purpose

Convert natural language into structured machine-understandable information.

Computers cannot directly understand questions.

They require structured analysis.

Input:

Why is Project Phoenix delayed?

Output:

```
JSON

{
  "intent": "RiskAnalysis",

  "entities": [
    "Project Phoenix"
  ],

  "department": "Engineering",

  "time": "Current"
}
```

Responsibilities

Intent Classification

Determine what the user wants.

Examples:

- General Search
- Timeline
- Ownership
- Risk Analysis
- Dependency Lookup
- Decision History
- Project Status

Multi-Intent Prediction

One question may contain multiple intents.

Example:

Compare Project Alpha and Project Beta risks.

Output:

Primary:

Comparison

Secondary:

RiskAnalysis

DependencyLookup

Confidence Scores

Each intent receives a probability.

Example:

RiskAnalysis

0.92

Timeline

0.71

Ownership

0.33

Planner uses these scores.

Entity Extraction

Identify:

- Projects
- Employees
- Teams
- Vendors
- Products

Example:

Project Phoenix

Legal Team

Vendor X

Relationship Extraction

Detect relationships.

Example:

Project Phoenix

↓

Blocked By

↓

Legal Approval

Time Scope

Examples:

- Last week
- Current
- Q1
- Last year

Department Detection

Identify:

- Finance
 - HR
 - Legal
 - Engineering
 - Sales
-

Output

QueryAnalysis

Python

Run

```
class QueryAnalysis:
```

```
    intent
```

```
    confidence
```

```
    entities
```

```
    relationships
```

```
    department
```

```
    time_scope
```

```
    retrieval_hints
```

6. Knowledge Execution Planner

Purpose

Planner decides HOW retrieval should happen.

It never retrieves information.

It creates a RetrievalPlan.

Input:

QueryAnalysis

↓

Output:

RetrievalPlan

Planner decides:

- Dense Retrieval?
- BM25?
- Graph Retrieval?
- Hop depth?
- Top-K?
- Latency budget?

Example

Question:

Who owns Project Phoenix?

Planner

Dense

✓

BM25

×

Graph

✓

Hop

2

TopK

30

RetrievalPlan

Python

Run

```
class RetrievalPlan:
```

```
    use_dense
```

```
    use_bm25
```

use_graph

hop_depth

top_k

latency_budget

7. Parallel Retrieval

Three retrieval systems execute simultaneously.

Dense Retrieval

Purpose:

Semantic search.

Uses embeddings.

Matches meaning rather than keywords.

Example:

Question

Vendor contract delay

Matches

Supplier agreement pending

Even without identical words.

Technology

- pgvector
 - HNSW Index
 - BGE embeddings
-

BM25 Retrieval

Purpose

Keyword search.

Best for:

- IDs
- Names
- Error codes
- Exact terminology

Example

Phoenix

Returns documents containing:

Phoenix

Knowledge Graph Retrieval

Purpose

Relationship traversal.

Instead of searching text,

it traverses graph edges.

Example

Project Phoenix

↓

Owned By

↓

Engineering

↓

Managed By

↓

Why Three Retrieval Systems?

Dense

Finds semantic meaning.

BM25

Finds exact keywords.

Graph

Finds relationships.

Together they provide much higher recall than any individual retriever.

Parallel Execution

Instead of

Dense

↓

BM25

↓

Graph

All execute concurrently.

Benefits:

- Lower latency
 - Better throughput
 - Improved scalability
-

8. Hybrid Fusion Engine

After retrieval,

three ranked lists exist.

Dense

↓

A B C

BM25

↓

B D E

Graph

↓

A D F

Need

One unified ranking.

Reciprocal Rank Fusion (RRF)

Documents appearing in multiple rankings receive higher scores.

Example

Document A

Dense Rank 1

Graph Rank 1

↓

Very high combined score.

Graph Priority

Structured graph facts may receive additional weighting.

Example

Knowledge Graph



Project blocked by Legal

is generally more trustworthy than an informal Slack message.

Output

One ranked evidence list.

9. Evidence Orchestrator

Fusion still produces individual evidence items.

Many describe the same event.

Example

Slack



Legal delay

Email



Legal delay

Meeting



Legal delay

Instead of three independent items,

Evidence Orchestrator creates

Evidence Group



Responsibilities

- Remove duplicates
 - Cluster semantically similar evidence
 - Preserve provenance
 - Aggregate facts
 - Build evidence groups
-

Evidence Group

Contains

- Summary
 - Sources
 - Confidence
 - Citations
-

10. Cross Encoder Reranker

Dense Retrieval uses approximate similarity.

Cross Encoder performs accurate ranking.

Input

Question

-

Evidence

↓

Transformer

↓

Relevance Score

Example

Question

Why is Project Phoenix delayed?

Evidence A

Legal approval delayed deployment.

Evidence B

Project completed after approval.

Dense Retrieval

A

0.91

B

0.89

Cross Encoder

A

0.99

B

0.18

Now ranking becomes much more accurate.

Common Models

- BGE-Reranker
 - Cohere Rerank
 - CrossEncoder MiniLM
-

Output

Top-ranked evidence groups.

11. Knowledge Synthesizer

Purpose

Convert evidence into structured knowledge.

Instead of returning documents,

it creates organizational understanding.

Responsibilities

Entity Resolution

Recognize

Phoenix

Project Phoenix

Phoenix Release

↓

Same entity.

Timeline Construction

Arrange events chronologically.

Example

Planning

↓

Vendor Selection

↓

Legal Review

↓

Approval Delay

↓

Deployment Delay

Decision Extraction

Extract

- Decision
- Owner
- Date
- Reason

Risk Extraction

Identify

- Vendor Risk
- Budget Risk
- Security Risk
- Compliance Risk

Relationship Building

Create graph links.

Example

Project

↓

Blocked By

↓

Legal Approval

Contradiction Detection

Slack

↓

Approved

Email

↓

Blocked

Conflict detected.

Derived Facts

Infer higher-level knowledge.

Example

Evidence

Vendor delay

↓

Legal waiting

↓

Deployment delayed

Derived Fact

Root Cause:

Vendor Contract Delay

Output

Structured knowledge

Python

Run

Knowledge

entities

relationships

timeline

decisions

risks

derived_facts

contradictions

Response 1 Summary

At the end of Response 1, Module 4 has transformed a natural language question into **structured organizational knowledge**, but it has **not yet verified its trustworthiness or packaged it for downstream modules**.

The pipeline completed so far is:

```
graph TD
    A[User Question] --> B[Policy Engine]
    B --> C[Query Intelligence]
    C --> D[Knowledge Execution Planner]
    D --> E["Parallel Retrieval  
(Dense + BM25 + Graph)"]
    E --> F[Hybrid Fusion Engine]
    F --> G[Evidence Orchestrator]
    G --> H[Cross Encoder Reranker]
    H --> I[Knowledge Synthesizer]
```

In **Response 2**, we'll complete Module 4 by covering:

1. Knowledge Validator
2. Knowledge Gap Detector
3. Evidence Quality Engine
4. Semantic Retrieval Cache
5. KnowledgeContext Packaging
6. Complete End-to-End Request Lifecycle
7. Module 4 Folder Structure
8. Production Best Practices
9. Complete Module 4 Summary and Architecture.